

Genetic Fuzzy Tree (GFT) for N-CMAPSS

Code documentation: `gft.py`, `ncmapss_features.py`, `gft_analysis.py`

1 Overview

The model is a *nested* Genetic Fuzzy Tree (GFT) trained **end-to-end** by a real-coded genetic algorithm (GA). It is a genuine tree, not a chain of independent stages: small sub-FIS (each 1–3 inputs $\rightarrow$ 1 output) are composed, and the *output of a child FIS is an input of its parent*.

- **Leaves — diagnosis.** One FIS per health modifier maps real cruise sensors to an estimated health parameter, `[sensors] -> theta_hat`. Leaves are *supervised*: each has a θ target, so $\hat{\theta}$ is directly comparable to ground truth.
- **Spool nodes — aggregation.** `hp` and `lp` fuse the two modifiers of each shaft into one *latent* damage value in $[0, 1]$. They have no target of their own.
- **Root — prognosis.** `RUL(hp, lp, age)` outputs the remaining useful life. Also latent-in, supervised-out.

The whole genome (all nodes at once) is optimised against a single scalar loss; the intermediate spool nodes are discovered by the GA.

Fuzzy model. Each FIS is a **zero-order Takagi–Sugeno** system on a *full grid* of rules. Inputs are fuzzified with a **Ruspini partition** (triangles that sum to 1 everywhere). With the product t-norm and the full grid, the rule firing strengths sum to exactly 1:

$$\sum_{\text{rules}} \prod_d \mu_{d,i_d}(x_d) = \prod_d \left(\sum_i \mu_{d,i}(x_d) \right) = 1,$$

so the Sugeno weighted average reduces to a plain dot product with the rule singletons c_r :

$$y(x) = \sum_r f_r(x) c_r.$$

Antecedent centres are fixed (data quantiles in z-space for sensors; the target’s quantiles or $[0, 1]$ for node inputs). The GA therefore optimises *only the singletons* — one scalar per grid cell — which keeps the model small, linear in its free parameters, and interpretable. The TSK order-1 mode, the roughness penalty and the monotone-consequent machinery of the previous version have been removed.

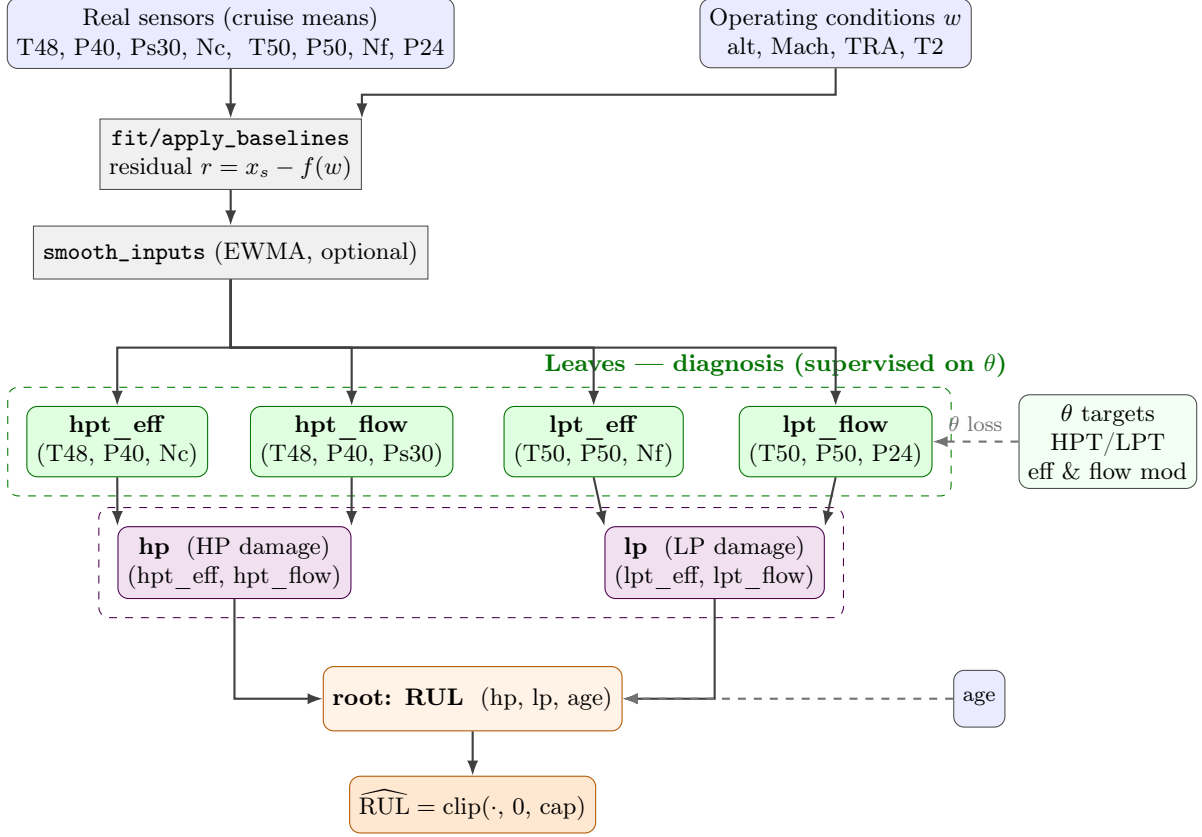
Hybrid loss. A single scalar drives the whole tree:

$$\mathcal{L} = \underbrace{\text{NASA}(\text{RUL})}_{\text{prognosis}} + w_\theta \cdot \underbrace{\frac{1}{L} \sum_\ell \frac{\text{RMSE}(\hat{\theta}_\ell, \theta_\ell)}{\text{range}(\theta_\ell)}}_{\text{magnitude}} + w_{\text{tr}} \cdot \underbrace{\frac{1}{L} \sum_\ell \text{mean}_{u \in \text{units}} (1 - \rho(\hat{\theta}_\ell, \theta_\ell)|_u)}_{\text{trend / shape}}$$

Both θ terms are **scale-free**: the magnitude term is divided by the target range, and the trend term is a Pearson correlation ρ computed *per unit*. Consequently, multiplying the sensors or the θ targets by any constant leaves the fit bit-identical (inputs are z-scored per node; the consequent box and the GA’s mutation step are both relative to the target range). The trend term exists because the plain RMSE is dominated by the per-cycle noise floor of a memoryless FIS and is therefore rather insensitive to whether the *shape* of the degradation is captured; $w_{\text{tr}} \sim 0.5\text{--}1.0$ rewards $\hat{\theta}$ *moving with* the truth regardless of scale.

Real sensors only. Only the measured sensors of C-MAPSS Table 2 (x_s : Wf Nf Nc T24 T30 T48 T50 P15 P21 P24 Ps30 P40 P50) are used. The virtual sensors of Table 3 (x_v : T40 P30 P45 W* epr Sm* NR* PCnFr phi) are not even loaded from the .h5 file.

2 Complete GFT tree



How to read it. The real sensors are first stripped of the dominant flight-condition effect ($x_s - f(w)$), optionally EWMA-smoothed along cycles, then fed to the four leaf FIS. Each leaf predicts one health modifier and is pulled by the θ loss. The two *latent* spool nodes fuse (*eff*, *flow*) into a single damage value per shaft, and the root FIS turns (*hp*, *lp*, *age*) into RUL. The tree is **symmetric by construction**: each shaft (HP = HPT; LP = Fan/LPC/LPT) has an efficiency leaf and a flow leaf feeding a global shaft-damage node. In DS02, HPT_flow happens to be constant; its leaf then simply predicts that constant (the tight consequent box keeps it noise-free), and the same tree still works on failure modes where HPT flow does move.

Editing the topology. The tree is a single module constant, TREE: a list of (name, [inputs], theta_target_or_None). An input is either a data column (a sensor, or "age") or the name of an earlier node. Children must precede parents, the root must be named "RUL", and a node takes at most 3 inputs. Per-input membership counts are given by writing an input as a pair:

```

TREE = [
  ("hpt_eff", [("T48", 5), ("P40", 3), "Nc"], "HPT_eff_mod"), # T48 -> 5 sets
  ("hpt_flow", ["T48", "P40", "Ps30"], "HPT_flow_mod"),
  ("lpt_eff", ["T50", "P50", "Nf"], "LPT_eff_mod"),
  ("lpt_flow", ["T50", "P50", "P24"], "LPT_flow_mod"),
  ("hp", [("hpt_eff", 2), ("hpt_flow", 2)], None),
  ("lp", ["lpt_eff", "lpt_flow"], None),
  ("RUL", ["hp", "lp", ("age", 6)], None),
]

```

A bare name uses the default `n_terms`. A node's rule count is the *product* of its inputs' set counts, so $(5, 3, 3) \Rightarrow 45$ rules — and 45 GA parameters for that node alone. Only put a θ target on a modifier that actually moves.

3 Reference: `gft.py`

1. Fuzzy inference

`_centers(x, n)`

n ordered Ruspini set-centres for one variable, placed at data quantiles so the fuzzy sets are populated. Guards against ties and near-constant inputs.

`_memberships(x, c)`

Triangular memberships, an $(n_{\text{samples}} \times n_{\text{terms}})$ matrix whose rows sum to 1. Each set is the piecewise-linear hat $0 \rightarrow 1 \rightarrow 0$; the two end sets are shouldered (flat 1 beyond the extreme centres), obtained for free by clamping in `np.interp`.

`_fis(cols, centers, singletons)`

One FIS: product t-norm firing over the full grid (C-order), then the Sugeno weighted average. The division by $\sum \text{firing}$ is a no-op under a Ruspini partition but stays safe if a centre set degenerates.

2. Tree construction

`_parse_input(inp, default)`

`("name", k) → (name, k)`; a bare `"name" → (name, default)`. This is what enables per-input membership counts.

`_leaf_sensors(tree)`

Sorted, de-duplicated list of the data columns the tree reads (i.e. everything that is not a node name and not `"age"`).

`_domain_centers(dom, k)`

k centres over a child node's output domain: the θ quantiles for a supervised leaf, $[0, 1]$ for a latent node. Lets a parent partition a child's output at its own resolution.

`build_tree(frame, tree=TREE, n_terms=3)`

Resolve every node's inputs against `frame` and *fix* the fuzzy centres (sensors: quantiles of the z-scored column, storing `mu/sd`; node inputs: the child's output domain). Inputs absent from the frame are silently dropped. Returns `meta`, a list of plain dicts `{name, inputs, n_rules, target, root}` — there is no `FISNode` class.

`n_params(meta)`

Total number of GA parameters (the sum of the nodes' rule counts, since only the singletons are free).

`genome_bounds(meta, frame, rul_hi, cons_pad=0.25)`

Per-node singleton box: the θ data range $\pm \text{cons_pad}$ -span for a supervised leaf, $[0, 1]$ for a latent spool node, $[0, \text{rul_hi}]$ for the root. A **(near-)constant target** gets a *tight* box around its mean instead of an inflated default — this is what stops a flat modifier (e.g. `HPT_flow` in DS02) from being filled with GA noise.

`predict_tree(frame, meta, genome)`

Evaluate the whole tree in topological order; returns `{node_name: output_array}`. Latent nodes are clipped to $[0, 1]$; leaves and the root are left on their natural scale.

3. Losses

`rmse(a, b)`

Root-mean-square error.

`nasa_score(y_true, y_pred)`

Asymmetric NASA/PHM prognostics score. Late predictions ($d > 0$, dangerous) are penalised harder ($/10$) than early ones ($d < 0$, safe, $/13$); zero at a perfect prediction. Minimising it makes the model prefer to under-predict RUL.

`_pearson(a, b)`

Pearson correlation; returns 0 if either series is (near-)constant.

`trend_penalty(pred, truth, unit_groups)`

Mean over units of $1 - \rho(\hat{\theta}, \theta)$ across that unit's cycles. *Scale-free* and noise-robust: it rewards $\hat{\theta}$ **moving with** the truth rather than matching its magnitude. Units whose true θ is constant carry no trend and are skipped; a flat prediction against a moving truth scores $\rho = 0$, i.e. the full penalty 1.

4. Genetic algorithm

`_tournament(fit, rng)`

Tournament selection, $k = 3$.

`genetic_optimize(loss, lo, hi, pop=80, gens=120, seed=0, x0=None, verbose=True, label="")`

Minimise `loss(genome)` over the box `[lo, hi]`. Tournament selection, BLX- α crossover, Gaussian mutation, elitism (2), and a geometrically cooling mutation step σ (floored at 0.05). Optionally seeded at `x0`. Returns (best genome, history).

5. Condition residuals and smoothing

`_design(frame, conds)`

Design matrix $[1, w, w^2]$ for the available operating conditions.

`_healthy_mask(frame)`

Rows to fit the healthy baseline on: pre-onset cycles if `since_onset` is known, otherwise each unit's first half of life.

`fit_baselines(frame, sensors, conds)`

Least-squares baseline coefficients per sensor, fit on healthy rows.

`apply_baselines(frame, coefs, conds)`

Replace each sensor by its residual $x_s - f(w)$ (in place, same column name). This strips the dominant flight-condition effect that would otherwise bury the small degradation term.

`smooth_inputs(frame, cols, span)`

EWMA-smooth the sensor columns along cycles *within* each unit; `span<=1` is a no-op. The FIS is *memoryless*, so per-cycle sensor noise passes straight into $\hat{\theta}$; smoothing the inputs here (identically at fit and predict time) is the direct cure for the cycle-to-cycle jitter.

6. Fit / predict

`fit_gft(frame, tree=TREE, n_terms=3, pop=80, gens=120, residualize=True, smooth_span=0, rul_cap=None, theta_weight=1.0, trend_weight=0.0, seed=0, verbose=True)`

Train the *whole* tree end-to-end on the hybrid loss of §1: NASA score on RUL, plus `theta_weight` $\times$ the range-normalised θ RMSE, plus `trend_weight` $\times$ the per-unit correlation penalty. `theta_weight=0` recovers pure RUL training; `trend_weight` ~ 0.5 – 1.0 sharpens trend fidelity. `rul_cap` applies the piecewise-linear (capped) RUL convention. Returns a plain-dict model (`tree`, `meta`, `genome`, `history`, `baselines`, `conds`, and the config echoes).

`predict_gft(frame, model)`

Full inference: $\text{sensors} \rightarrow \hat{\theta} \rightarrow (\text{hp}, \text{lp}) \rightarrow \text{RUL}$. Applies the same residualisation and smoothing as at fit time. Returns a frame with `unit`, `cycle`, one `<THETA>_hat` column per supervised leaf, one `<node>_h` column per latent node, and `RUL_hat` (clipped to `[0, rul_cap]`).

`inspect(model)`

Print, per FIS: its inputs, its target, its grid shape and the range of its learned singletons.

7. Self-test

`_synthetic_frame(units, cycles, seed)`

Signal-bearing synthetic frame in the extractor's schema: θ drifting after an onset, a per-flight operating point, real-sensor columns carrying a large condition effect plus a small damage effect, `RUL = EOL - cycle`.

`_self_test()`

Check the Ruspini partition of unity, train on the synthetic frame and print the train/test θ and RUL RMSEs against a mean baseline.

Module constants. `TREE` (the topology; see §2) and `CONDITIONS` (the operating conditions used for the healthy baseline).

4 Reference: `ncmapss_features.py`

Turns an N-CMAPSS `.h5` file into *one row per (unit, cycle)*. Because θ is imposed once per flight (constant within a cycle at 1 Hz), every feature is a cruise-mean aggregated to the cycle level. Columns keep their plain physical names (`T48`, `P40`, ...) so the tree can reference sensors directly.

`_names(arr)`

Decode a `*_var` group (byte-strings, possibly 2-D) into a flat list of clean Python strings.

`load_raw(path, split="dev")`

Load one split into a per-timestamp (1 Hz) DataFrame: `A` (unit, cycle, `Fc`, `hs`), `W` (conditions), `X_s` (real sensors), `T` (θ) and `Y` (RUL). **The virtual-sensor group `X_v` is deliberately not loaded**, so no virtual sensor can reach the model (and the real 5.3 M-row file uses less memory).

`cycle_features(path, split="dev", cruise_frac=0.95, min_cruise=30)`

The whole extractor. Keeps the high-altitude (cruise) part of each cycle (`alt >= cruise_frac * max(alt)`), averages the sensors and conditions over it, and returns one row per (unit, cycle) with: `unit`, `cycle`, `age` (1-indexed cycle count per unit), `since_onset` (cycles since the health flag `hs` flipped $0 \rightarrow 1$), `RUL`, the four operating conditions, the 13 real sensors and the θ modifiers. Cycles with fewer than `min_cruise` cruise samples are dropped.

`_make_synthetic_h5(path, ...) / _self_test()`

Build a small N-CMAPSS-like `.h5` (both `dev` and `test` groups) so the pipeline is testable without the real file.

Module constants. `CONDITIONS`; `SENSORS` — the 13 *real* measurements of Table 2 only; `THETA` — the health modifiers kept as targets.

5 Reference: `gft_analysis.py`

1. Utilities

`_color(u) / _grid(n)`

Per-unit colour (20-colour palette) and a compact subplot grid.

`_slice(meta, genome, node)`

The genome slice (the singletons) belonging to one node.

`_finish(fig, save, name, show)`

Save and/or display a figure, then close it.

`_prepare_dir(folder, clear=True)`

Create `folder` if it does not exist and **wipe the existing `.png`** so a rerun replaces stale figures instead of leaving orphans. Returns the filename prefix used by `_finish`.

2. Metrics and time-series plots

`print_metrics(feats, pred)`

Per-unit RUL RMSE and NASA score, plus an “ALL” row.

`plot_theta(feats, pred, ...)`

One figure per modifier: predicted vs actual θ against cycle, solid = real, dashed = predicted, one colour per unit, with that modifier’s RMSE in the title. This is the mid-node sanity check.

`plot_nodes(pred, ...)`

The *latent* spool health (`hp_h`, `lp_h`) vs cycle, one subplot per node, y clamped to $[0, 1]$.

`plot_rul(feats, pred, ...)`

RUL vs cycle, one subplot per unit: real (line) vs predicted (markers).

```
plot_scatter(feats, pred, ...)
```

Predicted vs true RUL, coloured per unit, with the $y = x$ line and the global RMSE in the title.

```
plot_fitness(model, ...)
```

The GA training curve: best loss per generation (single curve — the tree is trained in one pass, not per stage).

3. Control surfaces

```
_disp_range(inp) / _disp_axis_centers(inp)
```

Axis range and rule-grid centres in *display* units: raw (de-standardised) for sensors, native for $\theta/[0, 1]$ node inputs.

```
_eval_disp(node, Xdisp, singletons)
```

Evaluate one sub-FIS at display-unit points, standardising internally where required.

```
_surface(node, singletons, ix, iy, res=45)
```

Dense (XX, YY, ZZ) evaluation grid of a sub-FIS over inputs (ix, iy) , any other input frozen at the centre of its range.

```
plot_surface_node(node, singletons, kind="surface", res=45, ...)
```

One figure for one sub-FIS: a 1-D curve for a 1-input FIS, otherwise one panel per input *pair* (1 panel for 2 inputs, 3 for 3 inputs) side by side. `kind` $\in$ {"surface" (3-D), "contour" (top-down with the fuzzy-rule grid overlaid)}.

```
plot_surfaces(model, kind="surface", ...)
```

Draw *every* sub-FIS, each as its own figure (`surface_<node>.png`) — one giant grid of all nodes was unreadable.

4. Entry points

```
analyze(feats, pred, model=None, surfaces=False, surface_kind="surface", savedir=None, show=True)
```

Print the metrics and draw every figure in one call. If `savedir` is given, all figures are written to that folder (created if needed, existing `.png` replaced). Passing `model` also plots the GA curve and, with `surfaces=True`, the control surfaces.

```
save_figures(feats, pred, model=None, folder="figures", surfaces=True, surface_kind="contour")
```

Save every figure into folder with no on-screen display.

6 Typical usage

```
from ncmappss_features import cycle_features
from gft import fit_gft, predict_gft, inspect
from gft_analysis import analyze

feats = cycle_features("N-CMAPSS_DS02-006.h5", split="dev")

model = fit_gft(feats, gens=200,
                residualize=True,      # strip the flight-condition effect
                smooth_span=7,         # EWMA the sensors -> tames theta jitter
                theta_weight=1.0,      # magnitude term on theta
                trend_weight=0.8)      # shape term: reward tracking the trend
pred = predict_gft(feats, model)

analyze(feats, pred, model=model, surfaces=True, surface_kind="contour",
        savedir="figures")           # one fig per theta, one per sub-FIS
inspect(model)
```

Tuning notes.

- **Scaling the sensors or θ does nothing.** Inputs are z-scored per node and both θ loss terms are relative, so a $\times 100$ is a no-op (verified bit-identical). To make the loss more sensitive to a missed trend, raise `trend_weight`, not the units.
- **$\hat{\theta}$ jitter** is structural (the FIS is memoryless), not a lack of training: raise `smooth_span` rather than `gens`.
- **Rule-count blow-up:** a node's parameters = $\prod_d(\text{sets per input})$. Raise the per-input count only on inputs that carry real nonlinearity.

Functions prefixed with an underscore (`_...`) are internal; the others form each module's public API.